数据库设计与开发
Database Design & Development

计算机学院　单纯
sherryshan@bit.edu.cn
2024年3月

openGauss

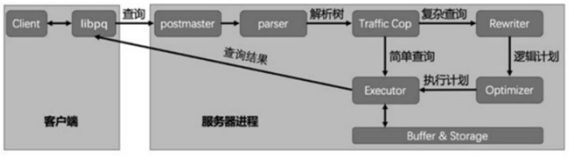
计算机学院　单纯
sherryshan@bit.edu.cn
2024年6月

内容概览

- 1 SQL引擎
- 2 查询优化
- 3 安全性机制
- 4 故障恢复系统
- 5 并发控制
- 6 其他

1 SQL引擎

- openGauss SQL响应流程



查询解析（parser）

- SQL解析对输入的SQL语句进行词法分析、语法分析、语义分析，获得查询解析树或者逻辑计划。
- 词法分析：从查询语句中识别出系统支持的各种字符，确定每个词的词性。
- 语法分析：使用词法分析中产生的词去匹配SQL语言的语法规则，如果一个SQL语句能够匹配一个语法规则，则生成对应的语法树。

查询解析（parser）

- 语义分析：检查语法树中对应的表、列、函数、表达式是否有对应的元数据描述，基于分析结果对语法树进行扩充生成查询树。

查询分流（traffic cop）

- traffic cop模块负责区分简单和复杂的查询指令。这种区分机制对简单命令执行最少的优化，将更多的时间投入到复杂命令上，减少处理时间。
  - 简单和复杂查询指令对应2类解析
    - 软解析（简单，旧查询）：当openGauss共享缓冲区中存在已提交SQL语句的解析表示形式，可复用这些内容执行语法和语义检查，避免查询优化的操作。

查询分流（traffic cop）

- 简单和复杂查询指令对应2类解析（续）
  - 硬解析（复杂，新查询）：当无缓存语句可复用、SQL语句第一次被加载到openGauss共享缓冲区中或一条在共享缓冲区中失效的语句被再次加载，则执行硬解析。共享Buffer的大小会影响解析调用的数量。

查询重写（rewrite）

- 利用已有语句特征和关系代数运算来生成更高效的等价语句，它遵循两个基本原则
  - 等价性：原语句和重写后的语句，输出结果相同。
  - 高效性：重写后的语句，比原语句在执行时间和资源使用上更高效。

查询重写（rewrite）

- openGauss查询重写的关键技术
  - 常量表达式化简：对常量表达式预先计算以提升效率。
  - 子查询提升：将子查询优化为“Semi Join”的形式。
  - 谓词下推：利用等价类推理（传递闭包）生成新的谓词条件，从而达到减小数据量和最大化利用索引的目的。

查询优化（optimizer）

- 为查询任务创建最佳执行计划
  - 生成查询计划：优化器生成查询中使用的每个单独关系（表）的计划。若查询需要连接两个或多个关系，在找到所有可行的单个关系的计划之后，将考虑连接关系的计划（嵌套循环连接、合并连接、哈希连接）。
  - 查询计划代价估计：对每条SQL语句生成的多个候选计划的执行代价进行计算，选择代价最小的计划。（总代价=IO代价+CPU代价）

查询执行（executor）

- 采用优化器创建的计划，并对其进行递归处理以提取所需的执行操作。这本质上是一种需求驱动的流水线执行机制。
- 执行器的处理控制流程
  - 创建查询描述。
  - 查询初始化：创建执行器状态（查询执行上下文）、执行节点初始化（创建表达式与每个元组上下文、执行表达式初始化）。

查询执行（executor）

- 执行器的处理控制流程（续）
  - 查询执行：执行处理节点（递归调用查询上下文、执行表达式，然后释放内存，重复操作）。
  - 查询完成：执行未完成的表格修改节点。
  - 查询结束：递归释放资源、释放查询及其子节点上下文。
  - 释放查询描述。

2 查询优化（1）

- 优化器的优化技术
  - RBO（Rule Based Optimization，基于规则的查询优化）
    - 根据预定义的启发式规则对SQL语句进行优化。
  - CBO（Cost Based Optimization，基于代价的查询优化）
    - 对SQL语句对应的待选执行路径进行代价估算，从待选路径中选择代价最低的执行路径作为最终的执行计划。
    - openGauss采用基于CBO的优化技术。

2 查询优化（2）

- 优化器的优化技术（续）
  - ABO（AI Based Optimization，基于机器学习的查询优化）
    - 收集执行计划的特征信息，借助机器学习模型获得经验信息，进而对执行计划进行调优，获得最优的执行计划。

2 查询优化（3）

- 在早期的数据库中，查询优化器通常采用启发式规则进行优化，这种优化方式不够灵活，往往难以获得最优的执行代价，而基于代价的优化则能够针对大多数场景高效筛选出性能较好的执行计划，但面对千人千面的用户和日趋复杂的实际查询场景，普适性的查询优化难以捕捉到用户特定的查询需求、数据分布、硬件性能等特征，难以全方位满足实际的优化需求。

2 查询优化（4）

- 近年来AI技术发展迅速，特别是在深度学习领域。ABO在建模效率、估算准确率和自适应性等方面都有很大优势，有望打破RBO和CBO基于静态模型的限制。通过对历史经验的不断学习，ABO将目标场景的模式进行抽象化，形成动态的模型，自适应地针对用户的实际场景进行优化。

3 安全性机制（1）

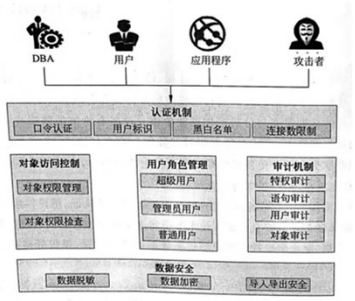
- 传统数据库系统
  - 构建于特定的操作系统平台上，以对外界提供数据服务或通过对接可视化管理界面提供数据库管理服务，整个系统部署在一个封闭的网络环境中。
  - 系统中的数据存放于物理存储介质上，存储介质可以为机械磁盘，也可以为SSD（固态硬盘）。硬件的稳定性和可靠性作为重要的一个环节，保障了数据整体的存储安全。

3 安全性机制（2）

- openGauss
  - 分布式系统，需要横跨不同的网络区域进行部署。除了需要像传统数据库那样从系统访问、数据导入导出、数据存储等维度来考虑系统安全体系外，还需要考虑网络安全、虚拟隔离等与实际业务场景紧密相关的安全措施。

3 安全性机制（3）

- openGauss安全机制体系



4 故障恢复系统

- openGauss的日志系统采用多个Log Writer（日志写盘）线程并行写的机制，充分发挥SSD（固态硬盘）的多通道IO能力。
- openGauss采用基于事务ID的多版本管理及历史版本的累积及清理方式，行存储引擎主要以Redo作为主要的手段，配以增量的检查点及日志并行回放，支持数据库的快速故障恢复。

5 并发控制（1）

- openGauss并发控制的核心是MVCC和快照机制
  - MVCC （多版本并发控制）的基本机制
    - 写事务不会原地修改元组内容，而是将被修改的元组标记为这条记录的一个旧版本，同时插入一条修改后的元组，从而产生这条记录的一个新版本；对于在一个查询事务开始时还没有提交的写事务，那么这个查询事务始终认为该写事务没有提交。

5 并发控制（2）

- openGauss并发控制的核心是MVCC和快照机制（续）
  - openGauss中有两种方式来实现快照
    - 活跃事务数组方法
      - 在数据库进程中，维护一个全局的数组，其中的成员为正在执行的事务信息，包括事务的事务号，该数组即活跃事务数组。
    - 时间戳方法
      - 在openGauss内部，使用一个全局自增的长整数作为逻辑的时间戳，模拟数据库内部的时序，该逻辑时间戳被称为提交序列号。每当一个事务提交的时候，在提交序列号日志中会记录该事务号对应的逻辑时间戳值。

5 并发控制（3）

- 使用MVCC和快照，可以有效解决读写冲突，使得并发的读事务工作在同一条元组的不同版本上，彼此不会互相阻塞。
- 对于并发的写事务，openGauss通过事务级别的锁机制来保证写事务的一致性和隔离性。

6 其他

- 01 openGauss和GaussDB(for openGauss)概述
- 02 云数据库介绍

参考资料

- 第7~11章，李国良 周敏奇编著，openGauss数据库核心技术，清华大学出版社，2020

Thank You